

NumberPlace.lispマニュアル

NumberPlace.lispはナンバープレース（数独）を理解し、解き方を習得するための機能を備えたプログラムです。プログラムはCommon Lispで記述されていてソースコードの状態で公開しています。

```
Enter Cell-Expression ("help"でセル式説明) : (or (row 3 6) (col 3 7))
#=====
# 1 2 . | 1 . . | B . . # . . . | . . . | . 2 . # . B . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | B . B # . 7 . | . 3 . | 4 . 6 # . B B | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . B . | . . . | . . . # . . . | . B . | . . . # . B . | . . . | . B B #
# B B B | . 8 . | B . B # . 9 . | B B . | . 1 . # . B B | . 7 . | . . B #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | B . B # . . . | . . . | . 2 . # . B . | . 2 3 | . . . #
# . 9 . | 4 . 6 | B . B # . 5 . | . 7 . | 4 . . # . . B | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . B | . . B | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . B #
# B B . | B . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#=====
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | 4 . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | B . . # . . . | . . . | . . . # . B . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | B . B # . 3 . | . 8 . | . 5 . # . . . | . . . | 4 . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # B . . | . . 9 | 7 . 9 #
#=====
Enter Cell-Expression ("help"でセル式説明) : █
```

本プログラムはフリーソフトウェアでありライセンスはGPL 3です。ライセンスの詳細は同梱の LICENSE.pdfをご覧ください。参考として「非公式GNU 一般公衆利用許諾書翻訳.pdf」も同梱しています。

本プログラムを利用するには、使用しているOS上で動作するCommon Lisp処理系が必要です。Linux各種ディストリビューション、Windows、macOS上でも動作するよう配慮していますが、検証できているのは下記の組み合わせです。

OS およびバージョン	CPU	処理系およびバージョン
Ubuntu 23.1	X86-64	CLISP 2.49.93+, SBCL 2.4.1
macOS 14.4	X86-64, ARM86-64	CLISP 2.49.93+, SBCL 2.4.1

特定のマシン、特定のOSに依存した機能はオプションとして分離しているので、正しく設定すれば上記以外の組み合わせでも動作するでしょう。特にWindows + WSL2(Ubuntu)は未検証ですが、特段の変更なく動作すると思われます。

Common Lisp処理系のダウンロード

CLISPの最新版は以下のリンクからダウンロードできます。

CLISP <http://clisp.cons.org/>

Steel Bank Common Lispの最新版は以下のリンクからダウンロードできます。

SBCL <https://www.sbcl.org/platform-table.html>

NumberPlace.lispのコンパイル

処理系をダウンロードしインストールしたら処理系を起動し、処理系のプロンプトから本プログラムをロードします。本プログラムを /home/daigo/Lisp/NumberPlace に置いているのであれば次のようにします。

```
> cd /home/daigo/Lisp/NumberPlace
> clisp ;; SBCLの場合は「sbcl」。
> (compile-file "NumberPlace.lisp")
> (load "NumberPlace.fas") ;; SBCLでは(load "NumberPlace.fasl")
```

コンパイル作業=(compile-file "NumberPlace.lisp")は最初の1回だけです。

2回目以降はコンパイル済みなので (load "NumberPlace.fas") または (load "NumberPlace.fasl")

のみです。上手く行かない場合はCLISPあるいはSBCLがパス名なしで実行可能なディレクトリに正しくインストールされているか、それぞれのコンパイラのインストール・マニュアルを確認して下さい。

また初期設定ファイルのサンプル「NumberPlace-init.lisp」も同梱していますが、このファイルはホームディレクトリに置いて下さい（NumberPlace.lispのソースコード内の設定を変えれば変更することもできます）。自分のホームディレクトリがどこかは

```
> cd
> pwd
```

とコマンドラインに2つのコマンドを入力すると「/home/daigo」などと表示される場所です。

「NumberPlace.lisp」は他のディレクトリに置いて、同じホームディレクトリに置いて大丈夫です。

CLISPはソースコードのコンパイルがSBCLより4～5倍速く、SBCLはコンパイル後の実行可能モジュールの実行速度がCLISPに比べて10倍程度高速です。

本プログラムの場合、CLISPでは外部コマンドの呼び出し機能も実装できていますが、SBCLは資料が乏しく、CLISPと同等には実装できていない部分があり一部制限があります。ただしCLISPでも特定の一箇所のみでカラー文字などの修飾文字出力が乱れる現象を確認しています。恐らくCLISP側のバグであり、NumberPlace.lispでは対処できていません。

	外部lessコマンド	外部lprコマンド	一般外部コマンド	環境変数取得	画面出力の保存
CLISP 2.49.93+	○	○ (要paps)	○	○	○
SBCL 2.4.1	×	○ (要paps)	△	○	×

SBCLの「一般外部コマンド」が△となっているのはlessやmoreなどの双方向入出力を行うコマンドが動作しないためです。catなど出力だけのコマンド呼び出しは問題ありません。「外部lessコマンド」呼び出しが×となっているのは、このためです。長いテキストもlessを使わずに出力していますが端末側のスクロール機能で代替可能です。

「外部lprコマンド」で「要paps」となっているのはUNIX/Linuxに標準で備わっているlprコマンドがutf-8フォントに対応していないためutf-8テキストをPostScriptファイルに変換する「paps」コマンドが必要なためです。paps コマンドを使わずにlprコマンドだけで日本語テキスト（utf-8）を印刷すると日本語部分が文字化けします。NumberPlace.lispはpapsコマンドが環境変数PATHで検索可能な範囲に存在すれば自動的に使用します。papsコマンドは

https://docs.oracle.com/cd/E75431_01/html/E71065/paps-1.html

から無料でダウンロードできます。インストール方法はaptコマンドが備わっている場合は

`sudo apt install paps` です。

CLISPの方が実行可能モジュールの実行速度が遅いとは言え、最も実行時間の掛かる部分の待ち時間でもCLISPが1.6秒程度、SBCLが0.16秒程度なので実使用上はどちらでも問題ない範囲です。

NumberPlace.lispの機能

機能は大きく分けて3つ

NumberPlace.lispの機能大別すると

- (1) 問題の解法過程を表示する機能。
- (2) 問題の解き筋を選択しながら検討する機能。

(3) 任意の盤面を使って各種手筋を習得できるようにするための機能。

の3つです。

(1)を実現しているのは「teach」関数です。盤面データを収めた変数名（Lispでは”シンボル”）を引数に取り、sample-board-5というシンボルに保存した問題（盤面データ）の解き筋の解説を見たい場合は処理系のプロンプトから

(teach sample-board-5)

と入力します。右の図のような解説盤面を表示しながら最終的な解答を表示します。

NumberPlace.lispでは

- ・基本手筋
- ・ローカライゼーション
- ・tuples(n国同盟=無制限)
- ・n-grid(x-wing, swordfish, jellyfish,...の一般形)
- ・配置確定法(Pattern Overlay Method)
- ・Nice Loop(連鎖セル数=無制限)
- ・Advanced Coloring(Simple Colors, Multi-Colorsを統合した手筋)
- ・Almost Locked Set(=ALS。X-Wing, XYZ-Wing, Sue De CoqなどWing系手筋の一般形)
- ・Grid-Based Almost Locked Set(=Sashimi Fish, Finned Fishなどの一般形)
- ・試行錯誤(仮置き)法

基本手筋です。[1]はブロック6では1の位置にしか置けません。
基本手筋です。[1]は9列目では1の位置にしか置けません。
==> [X]の位置から候補を削除できます。

#	1	2	.		1	.		1	.	#	.	.	.		.	.		2	.	#	2	.		2	.		2	.	#	
#	4	5	6		4	.	6		4	.	6	#	7	.		3	.		4	.	6	#	5	6		.	.		6	#
#	.	.	.		9	.		.	.	#	.	.	.		.	.		.	.	#	.	.		8	9		.	9	#	
#	.	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.	#	
#	2	.		.	.		.	.	#	.	.	.		2	.		2	.	#	.	.		.	.		.	.		2	#
#	5	6		3	.		7	.	#	8	.		5	.		6	#	1	.		4	.		.	6		.	6	#	
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	9		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	2	.		.	.		.	.	#	.	.	.		2	.		.	2	#	.	.		.	.		2	3	#		
#	4	5	6		8	.		4	.	6	9		4	5		1	.	#	5	6		7	.		.	6		.	6	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.		.	.		.	#
#	.	.		.	.		.	.	#	.	.	.		.	.		.	.	#	.	.		.	.						

ALS=使用, Nice Loop=連鎖セル数上限3で使用。

※ナンプレ超上級者向きの設定 = (machine-level)

localization=使用, n-grid=上限なしで使用, tuples=上限なしで使用, 配置確定法=使用,

Nice Loop=連鎖セル数上限なし。Advanced Coloring=使用。ALS=使用。cheat=許可。複数解=探索せず。

1コの解がありました。

```
#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 1 . | . 9 . | . 4 . # . 7 . | . 3 . | . 2 . # . 5 . | . 8 . | . 6 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 2 . | . 3 . | . 7 . # . 8 . | . 5 . | . 6 . # . 1 . | . 4 . | . 9 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 5 . | . 8 . | . 6 . # . 9 . | . 4 . | . 1 . # . 2 . | . 7 . | . 3 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 4 . | . 7 . | . 5 . # . 6 . | . 2 . | . 8 . # . 9 . | . 3 . | . 1 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 9 . | . 1 . | . 3 . # . 5 . | . 7 . | . 4 . # . 6 . | . 2 . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 8 . | . 6 . | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . 7 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 7 . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | . 4 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 6 . | . 4 . | . 1 . # . 3 . | . 8 . | . 5 . # . 7 . | . 9 . | . 2 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#-----#-----#
```

194732586237856149586941273475628931913574628862193457329417865758269314641385792

Evaluation took:

0.040 seconds of real time

0.029873 seconds of total run time (0.026132 user, 0.003741 system)

75.00% CPU

25 forms interpreted

82,099,824 processor cycles

15,695,328 bytes consed

試行錯誤関数の実行回数は0回, 探索深さの最大値は0でした。

手筋適用回数は4回, 平均難易度は3.00, 最大難易度は6, 合計難易度は12でした。

それぞれ(**novice-level**), (**middle-level**), (**senior-level**), (**advanced-level**), (**machine-level**)と入力すると、以後、設定した手筋と設定したレベルまでの手筋使用に限定して問題の解法手順解説を行います。”machine-level”が全ての手筋を制限なしで使用する設定です。

使用する手筋を限定すると試行錯誤法の介入が増えますが、試行錯誤法があるので複数解を含めて問題に解が存在する限り必ず解答を得ることができます。

現在の自分のレベルに合わせて設定することで、自分でも理解できる手筋だけからなる解法手順を見ることができます。

初期設定はNumberPlace-init.lisp内に記述されています。現在は「machine-level」が初期値となっています。変更したい場合はNumberPlace-init.lisp内の「(machine-level)」と書かれている行を見つけて、行頭に「; (セミコロン)」を加えてコメント・アウトして下さい。その後、望みのレベル設定関数（付近に記述されているはずです）の先頭に書かれているセミコロン全てを削除して下さい。**Common Lisp**では「; (セミコロン)」から行末までがコメント行となります。

問題の解答を表示したあとには、いくつかの情報も表示されます。

解答を表示する盤面の直後に表示されてい

る ” 194732586237856149586941273475628931913574628862193457329417865758269314641385792”
という81桁の数字は盤面の1行1列から9行9列までの各マスの数値を順に表わした数列です。インターネット上では候補数字が未確定のマスを「0（ゼロ）」で表した、この表現が使われていることがあります。NumberPlace.lispでは、この表現法を「chunk型」と呼び、内部処理で利用している2次元配列形式の盤面データとの相互変換をサポートしています。1行だけでデータをやり取りできるので便利です。chunk型データをNumberPlace.lispで読み書きする方法や内部形式との相互変換方法については後ほど記します。

その次には処理系によって表示方法は異なりますが実行時間や所要メモリ量が表示されます。

最後の2行はNumberPlace.lispが出力する統計情報で、試行錯誤法の実行回数、再帰的な探索深さの最大値、手筋の合計実行回数、NumberPlace.lispが独自に設定している各手筋の難易度に基づく情報です。難易度情報には確定した評価方法が定まっていないのであくまでも参考情報です。

NumberPlace.lispのソースコード内には30個ほどのナンバープレース問題がサンプルとして格納されています。試してみたい場合は問題を保存する適当な変数名（以下の例では”a”）を決めて

(setq a (select-game-from-memory :board t))

と実行して下さい。問題の概要を示す小さな盤面とともに問題群が表示され、番号で選べるようになっています。番号の右側にカッコで包まれて表示されている文字列は、保存されている本来の変数名です。

```

33 = (tuples-sample)
+-----+-----+-----+
| 5 4 + | 2 + = | = = 3 |
| = 2 + | + + + | + + 7 |
| = + 9 | 4 + 6 | 8 2 = |
+-----+-----+-----+
| 1 6 2 | 7 8 + | + 9 = |
| 4 9 3 | = = 2 | 7 8 6 |
| = 5 = | + 6 + | + = 2 |
+-----+-----+-----+
| 9 = 4 | + 2 1 | 6 7 = |
| 2 + + | 6 + + | + 3 + |
| 6 + + | + + 7 | 2 + 9 |
+-----+-----+-----+
Select number or name(中止は"quit") : 33
-----
tuples-sample
#2A((5 4 (1 6 7 8) 2 (1 7 9) (8 9) (1 9) (1 6) 3)
((3 8) 2 (1 6 8) (1 3 5 8 9) (1 3 5 9) (3 5 8 9) (1 4 5 9) (1 4 5 6) 7)
((3 7) (1 3 7) 9 4 (1 3 5 7) 6 8 2 (1 5))
(1 6 2 7 8 (3 4 5) (3 4 5) 9 (4 5))
(4 9 3 (1 5) (1 5) 2 7 8 6)
((7 8) 5 (7 8) (1 3 9) 6 (3 4 9) (1 3 4) (1 4) 2)
(9 (3 8) 4 (3 5 8) 2 1 6 7 (5 8))
(2 (1 7 8) (1 5 7 8) 6 (4 5 9) (4 5 8 9) (1 4 5) 3 (1 4 5 8))
(6 (1 3 8) (1 5 8) (3 5 8) (3 4 5) 7 2 (1 4 5) 9))
[6]> a
#2A((5 4 (1 6 7 8) 2 (1 7 9) (8 9) (1 9) (1 6) 3)
((3 8) 2 (1 6 8) (1 3 5 8 9) (1 3 5 9) (3 5 8 9) (1 4 5 9) (1 4 5 6) 7)
((3 7) (1 3 7) 9 4 (1 3 5 7) 6 8 2 (1 5))
(1 6 2 7 8 (3 4 5) (3 4 5) 9 (4 5))
(4 9 3 (1 5) (1 5) 2 7 8 6)
((7 8) 5 (7 8) (1 3 9) 6 (3 4 9) (1 3 4) (1 4) 2)
(9 (3 8) 4 (3 5 8) 2 1 6 7 (5 8))
(2 (1 7 8) (1 5 7 8) 6 (4 5 9) (4 5 8 9) (1 4 5) 3 (1 4 5 8))
(6 (1 3 8) (1 5 8) (3 5 8) (3 4 5) 7 2 (1 4 5) 9))

```

```

[7]> (print-normal a)
#=====#
# . . . | . . . | 1 . . # . . . | 1 . . | . . . # 1 . . | 1 . . | . . . #
# . 5 . | . 4 . | . . 6 # . 2 . | . . . | . . . # . . . | . . 6 | . 3 . #
# . . . | . . . | 7 8 . # . . . | 7 . 9 | . 8 9 # . . 9 | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . 3 | . . . | 1 . . # 1 . 3 | 1 . 3 | . . 3 # 1 . . | 1 . . | . . . #
# . . . | . 2 . | . . 6 # . 5 . | . 5 . | . 5 . # 4 5 . | 4 5 6 | . 7 . #
# . 8 . | . . . | . 8 . # . 8 9 | . . 9 | . 8 9 # . . 9 | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . 3 | 1 . 3 | . . . # . . . | 1 . 3 | . . . # . . . | . . . | 1 . . #
# . . . | . . . | . 9 . # . 4 . | . 5 . | . 6 . # . 8 . | . 2 . | . 5 . #
# 7 . . | 7 . . | . . . # . . . | 7 . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# . . . | . . . | . . . # . . . | . . . | . . 3 # . . 3 | . . . | . . . #
# . 1 . | . 6 . | . 2 . # . 7 . | . 8 . | 4 5 . # 4 5 . | . 9 . | 4 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # 1 . . | 1 . . | . . . # . . . | . . . | . . . #
# . 4 . | . 9 . | . 3 . # . 5 . | . 5 . | . 2 . # . 7 . | . 8 . | . 6 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # 1 . 3 | . . . | . . 3 # 1 . 3 | 1 . . | . . . #
# . . . | . 5 . | . . . # . . . | . 6 . | 4 . . # 4 . . | 4 . . | . 2 . #
# 7 8 . | . . . | 7 8 . # . . 9 | . . . | . . 9 # . . . | . . . | . . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# . . . | . . 3 | . . . # . . 3 | . . . | . . . # . . . | . . . | . . . #
# . 9 . | . . . | . 4 . # . 5 . | . 2 . | . 1 . # . 6 . | . 7 . | . 5 . #
# . . . | . 8 . | . . . # . 8 . | . . . | . . . # . . . | . . . | . 8 . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . . # . . . | . . . | . . . # 1 . . | . . . | 1 . . #
# . 2 . | . . . | . 5 . # . 6 . | 4 5 . | 4 5 . # 4 5 . | . 3 . | 4 5 . #
# . . . | 7 8 . | 7 8 . # . . . | . . 9 | . 8 9 # . . . | . . . | . 8 . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . 3 | 1 . . # . . 3 | . . 3 | . . . # . . . | 1 . . | . . . #
# . 6 . | . . . | . 5 . # . 5 . | 4 5 . | . 7 . # . 2 . | 4 5 . | . 9 . #
# . . . | . 8 . | . 8 . # . 8 . | . . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
nil

```

これで”a”という変数に33番に登録されていたナンバープレースの問題が保存されました。”#2a”で始まるカッコと数字の列は内部形式の2次元配列での盤面データそのものです。(print-normal a)と入力すると候補数字と確定値からなる盤面を人間向けの表示に直して表示します。

(teach a)

とすると途中経過が表示され最後に


```

1コの解がありました。
#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 5 . | . 4 . | . 8 . # . 2 . | . 7 . | . 9 . # . 1 . | . 6 . | . 3 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 6 . # . 8 . | . 1 . | . 5 . # . 9 . | . 4 . | . 7 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 7 . | . 1 . | . 9 . # . 4 . | . 3 . | . 6 . # . 8 . | . 2 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 1 . | . 6 . | . 2 . # . 7 . | . 8 . | . 3 . # . 5 . | . 9 . | . 4 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 4 . | . 9 . | . 3 . # . 1 . | . 5 . | . 2 . # . 7 . | . 8 . | . 6 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 8 . | . 5 . | . 7 . # . 9 . | . 6 . | . 4 . # . 3 . | . 1 . | . 2 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 9 . | . 3 . | . 4 . # . 5 . | . 2 . | . 1 . # . 6 . | . 7 . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 2 . | . 7 . | . 5 . # . 6 . | . 9 . | . 8 . # . 4 . | . 3 . | . 1 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 6 . | . 8 . | . 1 . # . 3 . | . 4 . | . 7 . # . 2 . | . 5 . | . 9 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====#
548279163326815947719436825162783594493152786857964312934521678275698431681347259
Real time: 1.437621 sec.
Run time: 1.437552 sec.
Space: 373729736 Bytes
GC: 144, GC time: 0.482947 sec.
試行錯誤関数の実行回数は0回，探索深さの最大値は0でした。
手筋適用回数は20回，平均難易度は7.60，最大難易度は10，合計難易度は152でした。
t

```

と解答が表示されます。途中経過はXterm 256またはANSI Color Terminal機能が有効で、NumberPlace.lispにどちらかの機能が有効であると設定していれば以下のようにカラー表示を使って分かりやすく表示します。これらのカラー表示機能を備えていない端末でもモノクロだけで判別できるように表示可能です。

次の図は途中経過表示のサンプルです。

Almost Locked Set(r#2)により [X] の位置から候補を削除できます。
 r2c1, r2c4, r2c5 の候補数字は Almost Locked Set [A]=(1 3 8 9) を構成しています。
 r8c5, r8c7 の候補数字は Almost Locked Set [B]=(1 4 9) を構成しています。
 Almost Locked Set[A] と [B] は候補数字(9)により相互にリンクし、共通の候補数字(1)が存在します。
 ==> r2c7 の [1] は Almost Locked Set 内のすべての [1] を見ることができるので削除できます。

```
#=====#
# . . . | . . . | 1 . . # . . . | 1 . . | . . . # 1 . . | . . . | . . . #
# . 5 . | . 4 . | . . . # . 2 . | . . . | . . . # . . . | . 6 . | . 3 . #
# . . . | . . . | 7 8 . # . . . | 7 . 9 | . 8 9 # . . 9 | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . B | . . . | . . . # B . B | B . B | . . . # X . . | . . . | . . . #
# . . . | . 2 . | . 6 . # . . . | . . . | . 5 . # 4 . . | 4 5 . | . 7 . #
# . B . | . . . | . . . # . B . | . . B | . 8 9 # . . 9 | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . 3 | 1 . . | . . . # . . . | 1 . 3 | . . . # . . . | . . . | 1 . . #
# . . . | . . . | . 9 . # . 4 . | . 5 . | . 6 . # . 8 . | . 2 . | . 5 . #
# 7 . . | 7 . . | . . . # . . . | 7 . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 1 . | . 6 . | . 2 . # . 7 . | . 8 . | . 3 . # . 5 . | . 9 . | . 4 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # 1 . . | 1 . . | . . . # . . . | . . . | . . . #
# . 4 . | . 9 . | . 3 . # . 5 . | . 5 . | . 2 . # . 7 . | . 8 . | . 6 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . . | . 5 . | . . . # . 9 . | . 6 . | . 4 . # . 3 . | . 1 . | . 2 . #
# 7 8 . | . . . | 7 8 . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . 3 | . . . # . . 3 | . . . | . . . # . . . | . . . | . . . #
# . 9 . | . . . | . 4 . # . 5 . | . 2 . | . 1 . # . 6 . | . 7 . | . 5 . #
# . . . | . 8 . | . . . # . 8 . | . . . | . . . # . . . | . . . | . 8 . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . # G . . | . . . | 1 . . #
# . 2 . | . . . | . 5 . # . 6 . | G . . | . 5 . # G . . | . 3 . | . . . #
# . . . | 7 8 . | 7 . . # . . . | . . G | . 8 9 # . . . | . . . | . 8 . #
#-----#-----#-----#-----#-----#
# . . . | 1 . 3 | 1 . . # . . 3 | . . 3 | . . . # . . . | . . . | . . . #
# . 6 . | . . . | . 5 . # . . . | 4 . . | . 7 . # . 2 . | 4 5 . | . 9 . #
# . . . | . 8 . | . . . # . 8 . | . . . | . . . # . . . | . . . | . . . #
#=====#
```

盤面データの入力方法は2つ

NumberPlace.lisp に盤面データを入力する方法は2つあります。

- ・ファイルに複数の問題を記述しておきファイル名を指定して読み込む方法。

- ・ NumberPlace.lispのプロンプトにしたがって手動で入力する方法。

ファイルに問題を記述する方法は2種類

- ・ NumberPlace.lispが内部形式として使用しているCommon Lispの2次元配列。
- ・ インターネット上で使用されている81桁の数字列の前後に「」（ダブルクオート）」を加えて文字列とした「chunk型」。

Common Lispの2次元配列は

```
> sample-board-5
#2A((0 0 0 0 3 0 0 0 0)
      (0 3 7 0 0 0 1 4 0)
      (0 8 0 9 0 1 0 7 0)
      (0 0 5 6 0 8 9 0 0)
      (9 0 0 0 7 0 0 0 8)
      (0 0 2 1 0 3 4 0 0)
      (0 2 0 4 0 7 0 6 0)
      (0 5 8 0 0 0 3 1 0)
      (0 0 0 0 8 0 0 0 0))
```

このような形式です。ナンバープレースの盤面表示にも近い形式です。2次元データ中の「0（ゼロ）」は確定値でない複数の候補数字が存在するセル（マス）であることを示しています。

81桁の数字列で表されるchunk型は以下のような形式です。上記の「sample-board-5」というシンボルに保存されているのと同じ盤面データを「board2chunk」という関数で変換したものです。

「chunk2board」という関数で逆変換が行えます。数字列中の「0（ゼロ）」は、やはり確定値でないセルであることを表しています。

```
> (board2chunk sample-board-5)
"000030000037000140080901070005608900900070008002103400020407060058000310000080000"
```

Common Lispの2次元配列形式のメリットは各セルの候補数字を具体的に指定できることです。次の例の「pm」という関数は「pencil-mark」という、盤面をペンシル・マーク形式に変換する関数の別名です。確定値に基づいて行・列・ブロック内から削除できる候補数字を取り除いた（以後、刈り込んだと表現します）結果を返します。例えば先頭行にある「(1 2 4 5 6)」というのは1行1列の候補数字が「1 2 4 5 6」だと言う意味です。確定値は単一の数字として独立しています。

```
> (pm sample-board-5)
#2A(((1 2 4 5 6) (1 4 6 9) (1 4 6) 7 3 (2 4 6) (2 5 6) (2 8 9) (2 6 9))
      ((2 5 6) 3 7 8 (2 5) (2 6) 1 4 (2 6 9))
      ((2 4 5 6) 8 (4 6) 9 (2 4 5) 1 (2 5 6) 7 (2 3 6))
      ((1 4 7) (1 4 7) 5 6 (2 4) 8 9 (2 3) (1 2 3 7))
      (9 (1 4 6) (1 3 4 6) 5 7 (2 4) (2 6) (2 3) 8)
      ((6 7 8) (6 7) 2 1 9 3 4 5 (6 7))
      (3 2 9 4 1 7 8 6 5)
      ((4 7) 5 8 2 6 9 3 1 (4 7))
      ((1 4 6 7) (1 4 6 7) (1 4 6) 3 8 5 (2 7) (2 9) (2 4 7 9)))
```

この内部表現の2次元配列を「print-normal」という関数で人間が読みやすい形式に変換して出力できます。

```
> (print-normal (pm sample-board-5))
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . 2 3 #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . 6 #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | 4 . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | 4 . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | 7 . 9 #
#=====#=====#=====#=====#=====#=====#=====#=====#
```

先程は説明を省略しましたが「ペンシル・マーク形式」というのは各候補数字を数字毎に固定位置に表示する形式です。1はセル内の左上、9であればセル内の右下です。候補数字を左上から詰めて表示することもできますがお薦めしません。手筋の解法手順表示ではペンシル・マーク形式限定で表示するものもあります（ユーザの設定は変更しません）。

Common Lispの2次元配列形式とchunk型のいずれの場合も、あらかじめ盤面データを保存するシンボル名を指定しておくことができます（指定がない場合は設定にしたがって自動的にシンボル名が与えられます）。

シンボル名を指定するにはCommon Lispの「setq」関数を使います。先程のsample-board-5の場合であればファイル内に

```
(setq sample-board-5
#2A((0 0 0 0 3 0 0 0 0)
    (0 3 7 0 0 0 1 4 0)
    (0 8 0 9 0 1 0 7 0)
    (0 0 5 6 0 8 9 0 0)
    (9 0 0 0 7 0 0 0 8)
    (0 0 2 1 0 3 4 0 0)
    (0 2 0 4 0 7 0 6 0)
    (0 5 8 0 0 0 3 1 0)
    (0 0 0 0 8 0 0 0 0))
)
```

と書きます。注意すべきなのは最後に閉じカッコ「)」が必要な点です。Common Lispではひとつの「式」は必ず対応するカッコで囲むことで「式」の範囲を確定しています。上記の例では「式」の始まりを表わすために「setq」の前に開きカッコ「(」が必要なので、最後にも対応する閉じカッコが必要です。「(setq シンボル名 2次元配列)」という形式です。Common Lispではアルファベットの大文字と小文字は区別されません。例えば「abc」と「ABC」「AbC」はすべて同じシンボル名として扱われます。もし、同じシンボル名が複数あれば最後のシンボル名に対するデータの割り当てが有効となります。

chunk型データの場合もまったく同様です。

```
(setq sample-board-5
"000030000037000140080901070005608900900070008002103400020407060058000310000080000"
)
```

と書きます。このようにして複数の問題を保管したファイル名が「my-file.lisp」であれば

(load-games "my-file.lisp")

と入力することでメモリ上に読み込めます。ファイル内の記述形式が2次元配列形式かchunk型かはNumberPlace.lispが自動的に判断して読み込みます。

メモリ上に読み込んだ問題の一覧を確認したい場合は処理系のプロンプトに

(select-game-from-memory :board t)

と入力します。

```
> (select-game-from-memory :board t)
```

```
-----
      0 = (adv-coloring-01)
```

```
+-----+-----+
| 2 5 = | 1 6 4 | = 7 9 |
| 6 = + | = = = | 1 = 4 |
| = 4 1 | + = = | 5 6 2 |
```

```
+-----+-----+
| = 1 = | 3 5 9 | + 2 = |
| 5 = + | 4 7 1 | + = 3 |
| + 3 + | 6 2 8 | = 5 1 |
```

```
+-----+-----+
| + 6 5 | + + = | 2 1 = |
| 1 = + | + + = | + + 6 |
| 8 9 + | = 1 6 | = 4 5 |
```

```
+-----+-----+
-----
```

```
      1 = (adv-coloring-02)
```

```
(途中省略)
```

```
-----
      33 = (tuples-sample)
```

```
+-----+-----+
| 5 4 + | 2 + = | = = 3 |
| = 2 + | + + + | + + 7 |
| = + 9 | 4 + 6 | 8 2 = |
```

```
+-----+-----+
| 1 6 2 | 7 8 + | + 9 = |
| 4 9 3 | = = 2 | 7 8 6 |
| = 5 = | + 6 + | + = 2 |
```

```
+-----+-----+
| 9 = 4 | + 2 1 | 6 7 = |
| 2 + + | 6 + + | + 3 + |
| 6 + + | + + 7 | 2 + 9 |
```

```
+-----+-----+
-----
```

```
Select number or name(中止は"quit") : 33
```

```
-----
tuples-sample
```

```
#2A((5 4 (1 6 7 8) 2 (1 7 9) (8 9) (1 9) (1 6) 3)
  ((3 8) 2 (1 6 8) (1 3 5 8 9) (1 3 5 9) (3 5 8 9) (1 4 5 9) (1 4 5 6) 7)
  ((3 7) (1 3 7) 9 4 (1 3 5 7) 6 8 2 (1 5))
  (1 6 2 7 8 (3 4 5) (3 4 5) 9 (4 5))
  (4 9 3 (1 5) (1 5) 2 7 8 6)
  ((7 8) 5 (7 8) (1 3 9) 6 (3 4 9) (1 3 4) (1 4) 2)
  (9 (3 8) 4 (3 5 8) 2 1 6 7 (5 8))
  (2 (1 7 8) (1 5 7 8) 6 (4 5 9) (4 5 8 9) (1 4 5) 3 (1 4 5 8))
  (6 (1 3 8) (1 5 8) (3 5 8) (3 4 5) 7 2 (1 4 5) 9))
```

```
> tuples-sample
```

```
#2A((5 4 (1 6 7 8) 2 (1 7 9) (8 9) (1 9) (1 6) 3)
  ((3 8) 2 (1 6 8) (1 3 5 8 9) (1 3 5 9) (3 5 8 9) (1 4 5 9) (1 4 5 6) 7)
  ((3 7) (1 3 7) 9 4 (1 3 5 7) 6 8 2 (1 5))
  (1 6 2 7 8 (3 4 5) (3 4 5) 9 (4 5))
  (4 9 3 (1 5) (1 5) 2 7 8 6)
  ((7 8) 5 (7 8) (1 3 9) 6 (3 4 9) (1 3 4) (1 4) 2)
  (9 (3 8) 4 (3 5 8) 2 1 6 7 (5 8))
  (2 (1 7 8) (1 5 7 8) 6 (4 5 9) (4 5 8 9) (1 4 5) 3 (1 4 5 8))
  (6 (1 3 8) (1 5 8) (3 5 8) (3 4 5) 7 2 (1 4 5) 9))
```

初期状態では30個ほどの問題がサンプルとして含まれています。ファイルから読み込んだ問題もサンプル問題に追加される形で表示されます。上記の例では33番の問題を選んでいきます（青字部分）。与えられているシンボル名が「tuples-sample」で盤面データの内容が表示されています。

メニューに表示されている小さな盤面はセルの内容が確定値なら確定値の数字、候補数字が2個のセルであれば「=」、候補数字が3個以上あるセルであれば「+」が表示されています。

詳しく表示できる大きな盤面を表示する場合は

(print-normal tuples-sample)

とします。

```
> (print-normal tuples-sample)
#=====#
# . . . | . . . | 1 . . # . . . | 1 . . | . . . # 1 . . | 1 . . | . . . #
# . 5 . | . 4 . | . 6 # . 2 . | . . . | . . . # . . . | . . 6 | . 3 . #
# . . . | . . . | 7 8 . # . . . | 7 . 9 | . 8 9 # . . 9 | . . . | . . . #
#-----#-----#-----#-----#
# . . 3 | . . . | 1 . . # 1 . 3 | 1 . 3 | . . 3 # 1 . . | 1 . . | . . . #
# . . . | . 2 . | . . 6 # . 5 . | . 5 . | . 5 . # 4 5 . | 4 5 6 | . 7 . #
# . 8 . | . . . | . 8 . # . 8 9 | . . 9 | . 8 9 # . . 9 | . . . | . . . #
#-----#-----#-----#-----#
# . . 3 | 1 . 3 | . . . # . . . | 1 . 3 | . . . # . . . | . . . | 1 . . #
# . . . | . . . | . 9 . # . 4 . | . 5 . | . 6 . # . 8 . | . 2 . | . 5 . #
# 7 . . | 7 . . | . . . # . . . | 7 . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . 3 # . . 3 | . . . | . . . #
# . 1 . | . 6 . | . 2 . # . 7 . | . 8 . | 4 5 . # 4 5 . | . 9 . | 4 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#
# . . . | . . . | . . . # 1 . . | 1 . . | . . . # . . . | . . . | . . . #
# . 4 . | . 9 . | . 3 . # . 5 . | . 5 . | . 2 . # . 7 . | . 8 . | . 6 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#
# . . . | . . . | . . . # 1 . 3 | . . . | . . 3 # 1 . 3 | 1 . . | . . . #
# . . . | . 5 . | . . . # . . . | . 6 . | 4 . . # 4 . . | 4 . . | . 2 . #
# 7 8 . | . . . | 7 8 . # . . 9 | . . . | . . 9 # . . . | . . . | . . . #
#-----#-----#-----#-----#
# . . . | . . 3 | . . . # . . 3 | . . . | . . . # . . . | . . . | . . . #
# . 9 . | . . . | . 4 . # . 5 . | . 2 . | . 1 . # . 6 . | . 7 . | . 5 . #
# . . . | . 8 . | . . . # . 8 . | . . . | . . . # . . . | . . . | . 8 . #
#-----#-----#-----#-----#
# . . . | 1 . . | 1 . . # . . . | . . . | . . . # 1 . . | . . . | 1 . . #
# . 2 . | . . . | . 5 . # . 6 . | 4 5 . | 4 5 . # 4 5 . | . 3 . | 4 5 . #
# . . . | 7 8 . | 7 8 . # . . . | . . 9 | . 8 9 # . . . | . . . | . 8 . #
#-----#-----#-----#-----#
# . . . | 1 . 3 | 1 . . # . . 3 | . . 3 | . . . # . . . | 1 . . | . . . #
# . 6 . | . . . | . 5 . # . 5 . | 4 5 . | . 7 . # . 2 . | 4 5 . | . 9 . #
# . . . | . 8 . | . 8 . # . 8 . | . . . | . . . # . . . | . . . | . . . #
#=====#
```

このまま問題の解法過程を確認したいのであれば

(teach tuples-sample)

とします。解法過程で使用する手筋やレベルを変更したい場合は処理系のプロンプトに

(novice-level)

などと入力すれば設定を変更できます。再度設定を変更しない限り、変更した設定が有効です。ただし、初期設定ファイルを読み込むと、初期設定ファイル内に存在する設定で上書きされるのを忘れないで下さい。恒久的に変更する場合は初期設定ファイル内の設定を書き換えます。

1. 手動入力方法

ナンバープレースの問題を手動で入力することもできます。9x9サイズのナンバープレースの場合、1行1列から9行9列までの計81個のセルすべての値を入力します。行ごとに入力していく「enter-board」という関数を使います。enter-board関数は入力された問題の盤面データを表わすCommon Lispの2次元配列データを返します。「sample-board-a」というシンボルに手動でデータを入力するには次のようにします。

```
(setq sample-board-a (enter-board))

> (setq sample-board-a (enter-board))
行ごとに入力します。空欄には「0」を入力してください。
1行目: 0 0 0 0 3 0 0 0 0
2行目: 0 3 7 0 0 0 1 4 0
3行目: 0 8 0 9 0 1 0 7 0
4行目: 0 0 5 6 0 8 9 0 0
5行目: 9 0 0 0 7 0 0 0 8
6行目: 0 0 2 1 0 3 4 0 0
7行目: 0 2 0 4 0 7 0 6 0
8行目: 0 5 8 0 0 0 3 1 0
9行目: 0 0 0 0 8 0 0 0 0
#2A((0 0 0 0 3 0 0 0 0)
     (0 3 7 0 0 0 1 4 0)
     (0 8 0 9 0 1 0 7 0)
     (0 0 5 6 0 8 9 0 0)
     (9 0 0 0 7 0 0 0 8)
     (0 0 2 1 0 3 4 0 0)
     (0 2 0 4 0 7 0 6 0)
     (0 5 8 0 0 0 3 1 0)
     (0 0 0 0 8 0 0 0 0))
> sample-board-a
#2A((0 0 0 0 3 0 0 0 0)
     (0 3 7 0 0 0 1 4 0)
     (0 8 0 9 0 1 0 7 0)
     (0 0 5 6 0 8 9 0 0)
     (9 0 0 0 7 0 0 0 8)
     (0 0 2 1 0 3 4 0 0)
     (0 2 0 4 0 7 0 6 0)
     (0 5 8 0 0 0 3 1 0)
     (0 0 0 0 8 0 0 0 0))
```


青字部分がユーザが入力した部分です。プロンプトには「空欄には「0」を入力して下さい」とありますが、候補数字が分かっているのであればカッコで囲って直接入力することもできます。1行1列に候補数字1, 2, 4, 5, 6と入力したいのであれば1行1列の入力欄に「0」を書く代わりに「(1 2 4 5 6)」と書きます。もっとも、初期状態の各セルの候補数字は「pm」関数が自動的に刈り込みを行ってくれるので、通常は不要です。

```
> (print-normal (pm sample-board-a))
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . 2 3 #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . 6 #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#=====#=====#=====#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | 4 . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | 4 . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | 7 . 9 #
#=====#=====#=====#=====#
```

問題の解き筋を選択しながら検討する

NumberPlace.lispの2つ目の機能は、複数ある問題の解き筋をユーザが自身の判断で解き筋を決めながら検討する機能です。

一般的にナンバープレースのひとつの盤面で成立する手筋は複数あります。ひとつの手筋に対しても複数ヶ所で同じ手筋が成立している場合も普通にあります。各盤面ごとに複数の手筋が成立するので、どの手筋を選んで手を進めるかという選択肢は、ほとんど無数と言っても良いくらい大量にあります。

NumberPlace.lispでは、無数に存在する解き筋を検討するために各盤面において選択可能な手筋を示し、選択的に手を進めることができるようにしています。また、任意の時点で任意の盤面まで戻って、別の解き筋を調べることもできます。このための機能を提供するのが関数「`examin`」です。`examin`により、自動的にすべての解き筋を網羅的に調べ上げることも可能ですが、分岐する盤面数が数千～数万に及び数十時間の実行時間と膨大なメモリが必要となる場合があるので、あまりお勧めできません。2024年時点の高速なパソコンより100倍速く、100倍のメモリを備えたパソコン上での実行をお勧めします。このようなケースでは最低でも64GBの実メモリを備えたパソコンでないと実用的ではありません。

関数`examin`の実行方法は3種類

関数`examin`を実行するパターンは次の3種類です。

- ・ 引数なしで実行する。
- ・ 問題を保存したファイル名を引数として実行する。
- ・ 問題を保存したシンボル名を引数として実行する。

いずれの場合でも最終的にはひとつのシンボルに保存されたひとつの問題を対象として設定した上で検討が始まります。

まず、引数なしで実行する場合から紹介します。引数なしで実行するといくつかのプロンプトが表示されてユーザに選択を求めます。

最初のプロンプトは

メモリ上にロード済みの盤面データから選びますか。 (y or n)

というものです。「y」と入力後に改行をタイプすると、メモリ上にロードされている問題の一覧が表示されて、その中から番号または番号と共にカッコに囲まれて表示されているシンボル名を入力することで対象とする問題を選択します。上記のプロンプトに「n」と入力すると

数独問題が格納された変数名、あるいはファイル名を入力して下さい。
[retun]のみを入力すると盤面データの手入力となります("quit"で実行中止)。

というメッセージと問題が格納された変数名（シンボル名）、またはファイル名（ダブルクォートで囲まれた文字列）の入力を求めるプロンプトが表れます。入力がシンボルであれば、メモリ上に一致する盤面データを持つシンボルが存在するか調べ、存在しなければ入力された文字列に一致するファイル名がないかを調べ、そのファイルからのデータ読み込みを試します。ファイルが存在しないか保存すべき盤面データが含まれていなければ「nil」という値を返し何もしません。その場合は、正しいファイル名、あるいは正しいデータが含まれているファイルかどうかを確認してexaminコマンドの入力からやり直して下さい。

上記のメニューに表示されている通り、リターン・キーのみを入力すると前述のenter-board関数での盤面データの手動入力となります。「quit」と入力すると関数examinの実行を中止して処理系のプロンプトに戻ります。

ファイル名は文字列として与える

関数examinの2番めの実行方法は引数として盤面データが保存されているファイル名を指定する方法です。ファイル名はユーザが入力した通りの名前で検索します。ファイル名に対して大文字・小文字を区別するOSでは大文字・小文字の違いを含めた文字列がファイル名と違っていれば一致するファイル名とは見なされません。ファイル名は「」（ダブルクォート）で囲まれた文字列として指定します。

Top-95.txtというファイルに目的のナンバープレースの問題が保存されているのであれば

(load-games "Top-95.txt")

と入力します。ファイル名の先頭の「t」が大文字であれば、必ず大文字の「T」から書き始めます。NumberPlace.lispはTop-95.txt内に含まれていたすべての問題を表示して問題を選ぶよう求めます。ファイルが存在しなければ「nil」という値を返して何もせず処理系のプロンプトに戻ります。

盤面データを保存したシンボル名を与える

最後は関数examinの引数として盤面データを保存したシンボル名を与える方法です。「sample-board-5」というシンボルに盤面データが保存されているのであれば

(examin sample-board-5)

と入力します。sample-board-5に正しい盤面データ型の値が保存されていないならば(examin sample-board-5)は単に「nil」という値だけを返して何もしません。

引数のシンボルに何の値も保管されていないと処理系からのエラーメッセージが表示されて、(CLISPとSBCLの場合は) 処理系のデバグが起動し、デバグのプロンプトが表示されます。例えば何の値も保存していない「b」というシンボルを引数として実行すると

CLISPの場合は

```
> (examin b)

*** - system::read-eval-print: variable b has no value
The following restarts are available:
USE-VALUE      :R1      Input a value to be used instead of b.
STORE-VALUE    :R2      Input a new value for b.
ABORT          :R3      Abort main loop
```

と表示されます。この場合は「:r3」と入力してリターンキーをタイプすると処理系の（通常の）プロンプトに戻れます。

SBCLの場合は

```
* (examin b)

debugger invoked on a UNBOUND-VARIABLE @547A60A4 in thread
#<THREAD tid=8441 "main thread" RUNNING {1067A98133}>:
  The variable B is unbound.

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE   ] Retry using B.
  1: [USE-VALUE  ] Use specified value.
  2: [STORE-VALUE] Set specified value and use it.
  3: [ABORT      ] Exit debugger, returning to top level.

(SB-INT:SIMPLE-EVAL-IN-LEXENV B #<NULL-LEXENV>)
0]
```

と表示されるので「3」と入力してリターンキーをタイプすれば処理系のプロンプトに戻れます。他の処理系でも同様だと思います。処理系の表示するメッセージにしたがってエラー処理状態から復帰して下さい。

関数examinのメニュー表示は4段階

関数examinに前述のいずれかの方法で盤面データを与えると、関数examinは与えられた盤面データを対象とした各種処理を行う機能をメニュー形式で提供します。メニューは

- | | |
|--------------------|--------------|
| ・すべてのメニュー項目を表示 | (Verboseモード) |
| ・通常必要とされるメニュー項目を表示 | (Normalモード) |
| ・最小限のメニュー項目を表示 | (Minimumモード) |
| ・一切のメニュー項目の表示なし | (Commandモード) |

の4段階の表示量を選択できます。既定値はNormalモードです。メニュー項目に表示されていない項目

Verboseモード

I)nformation	現在の盤面と付随する情報を表示します。
P)ause(nil)	指定した盤面数を出力するごとに一時停止します。
T)esuji	現在の盤面に適用可能な手筋一覧を表示します。
D)escription	指定されたルート番号の解法過程を表示します。 カレント・ノードが指定されたルート番号のノードに変わります。 数値を直接入力することも出来ます。
G)uess	検討中の盤面を使って手筋習得練習を行います。
G)o)al	解決済みの解き筋を表示します。
R)o)ute	現時点までに判明している解き筋を表示します。
E)x)plore	全ての解法過程ルートを探索します。一時停止設定も可。
A)uto save(5)	解法過程の途中結果を設定分数ごとにファイルに保存します。 Exploreコマンドでの一時停止設定が優先します。
B)o)ard	候補数字ありの盤面と、なしの小さな盤面を切り替えます。
F)ind	指定されたノード番号/ラベルを持つノードに移動します。
C)h)ange	現在のノードのラベルを変更します。
U)p	親ノードに移動します。
S)a)ve	現時点までの解法経路情報を保存します。
L)o)ad	ファイルから解法経路情報を読み込みます。 最後に読み込んだ解法経路情報を関数[examin]のデータとして設定します。 検討を中断した状態から再開できます。
O)utput	現時点までの解法ルート図(テキスト)を保存します。 解法ルート情報が存在しなければ保存用ファイルも作成されません。
S)e)lect Game	数独問題を選択します。数独問題群のリセットもここ。
E)n)ter Game	数独問題を新規に手入力します。
S)t)ore Games	メモリ上の全ての数独の問題をファイルに保存します。
R)e)ad games	ファイルから数独の問題を読み込みます。 名前を指定していない数独問題に半自動で名前を付けます。
C)o)llection	現在の盤面をファイルに追記します。
L)p)r	指定するノード番号の盤面を印刷します。
L)e)vel	使用する手筋と手筋に対する制限を5段階から選びます。
M)e)nu	メニューの表示量を3段階で選びます。現在はVerboseです。
E)v)al	設定変更関数などを評価する。
Q)uit	検討を終了します。
H)elp	短い手筋解説を表示します。
V)ersion	実行中のNumberPlace.lispのバージョンを表示します。

であっても、項目名を入力すれば該当機能を実行できます。メニュー項目の表示量を減らすと画面内に他の情報をより多く表示できます。各モードでのメニュー表示は以下のようになります。

Normalモード

I)nformation	現在の盤面と付随する情報を表示します。
P)ause(nil)	指定した盤面数を出力するごとに一時停止します。
T)esuji	現在の盤面に適用可能な手筋一覧を表示します。
D)escription	指定されたルート番号の解法過程を表示します。 カレント・ノードが指定されたルート番号のノードに変わります。
Gu)ess	検討中の盤面を使って手筋習得練習を行います。
Ro)ute	現時点までに判明している解き筋を表示します。
Ex)plore	全ての解法過程ルートを探索します。一時停止設定も可。
A)uto save(5)	解法過程の途中結果を設定分数ごとにファイルに保存します。
B)oard	候補数字ありの盤面と、なしの小さな盤面を切り替えます。
F)ind	指定されたノード番号/ラベルを持つノードに移動します。
O)utput	現時点までの解法ルート図(テキスト)を保存します。
Se)lect Game	数独問題を選択します。数独問題群のリセットもここ。
En)ter Game	数独問題を新規に手入力します。
St)ore Games	メモリ上の全ての数独の問題をファイルに保存します。
Re)ad games	ファイルから数独の問題を読み込みます。
Co)llection	現在の盤面をファイルに追記します。
Lp)r	指定するノード番号の盤面を印刷します。
Me)nu	メニューの表示量を3段階で選びます。現在はNormalです。
Ev)al	設定変更関数などを評価する。
Q)uit	検討を終了します。

Minimumモード

T)esuji	現在の盤面に適用可能な手筋一覧を表示します。
D)escription	指定されたルート番号の解法過程を表示します。 カレント・ノードが指定されたルート番号のノードに変わります。
Gu)ess	検討中の盤面を使って手筋習得練習を行います。
Ro)ute	現時点までに判明している解き筋を表示します。
Ex)plore	全ての解法過程ルートを探索します。一時停止設定も可。
A)uto save(5)	解法過程の途中結果を設定分数ごとにファイルに保存します。
B)oard	候補数字ありの盤面と、なしの小さな盤面を切り替えます。
F)ind	指定されたノード番号/ラベルを持つノードに移動します。
Me)nu	メニューの表示量を3段階で選びます。現在はMinimumです。

Commandモード

再度メニューを表示するには "Menu[return]" と入力します。
Verbose表示モードで表示される全てのメニュー・コマンドが使えます。

初回のみ上記のようなメッセージが表示されますが、以降は一切のメニュー表示なしです。

プロンプトには実行中の処理系名と時刻が表示されます。

SBCLの場合

*(SBCL/X86-64 17:44:51) Select menu :

CLISPの場合

*(CLISP/X86_64 17:53:24) Select menu :

メニュー項目実行はメニュー名を入力

メニューには途中に「)」が挿入されたメニュー項目名とメニュー項目の説明が表示されています。メニュー項目を実行するには途中の「)」を除いたフルスペルのメニュー項目名を入力するか、「)」の直前までの短縮メニュー項目名を入力します。

「現在の盤面に適用可能な手筋一覧を表示します。」とある「M)ethods」コマンドを実行するには「m)」のみを入力してリターンキーをタイプするか「methods)」とフルスペルを入力してリターンキーをタイプします。大文字・小文字は区別しません。大文字で入力しても大文字と小文字が混ざった入力でも構いません。ただし「meth)」や「method)」など、短縮名でもフルスペルでもない入力は正しいメニュー項目名とは解釈しません。

メニュー項目の表示量を変更するコマンドは「Me)nu)」です。menuと入力すると

メニューの表示量を次の3段階から選んで下さい。

```
V)erbose : 詳細
N)ormal  : 通常
M)inimum : 最小
C)ommand : なし
Select V/N/M/C : v
```

と表示されます。この場合、通常は「)」の前の1文字だけ入力すれば十分ですが、もちろんフルスペルで入力しても構いません。またメニュー項目のコマンド名を入力したあとに、更にサブメニューがあることを知っているのであればサブメニューのコマンド名をまとめて指定することも可能です。

メニュー項目量を選択する「Me)nu)」コマンドで、「N)ormal)」モードを選択する場合

menu normal あるいは me n

とまとめて入力することもできます。これは他のメニュー項目でも同様です。

盤面ごとに解き筋を選択できるMethodsメニュー

examin関数の実行が始まると与えられた盤面に対して刈り込みを行った初期盤面を表示し、その後にメニューが表示されます。処理系によっては使用できない機能があり、そのような機能はメニュー項目に表示されません。このため処理系によってメニュー項目数が異なる場合があります。うかつに実行すると非常に長い実行時間や大量のメモリが必要となる機能は設定によって隠されている場合もあります。デバッグ用として実装した機能なども設定しない限り見えないようになっています。

主要機能はmethodsコマンドとguessコマンド

以上の理由から処理系と設定によってメニュー数は変わりますが、Verboseモードでは概ね30前後のメニュー項目が表示されます。これらのメニュー項目の中で主要機能となるのは「M)ethods」コマンドと「G)uess」コマンドです。

「M)ethods」コマンド（以下、methodsコマンド）は現在の盤面に対して適用可能な手筋一覧を表示し、どのルートに進むかを選択できる機能を提供するコマンドです。例えば

+	-	-	-	+	-	-	-	+	-	-	-	+
	-	-	-		-	3	-		-	-	-	
	-	3	7		-	-	-		1	4	-	
	-	8	-		9	-	1		-	7	-	
+	-	-	-	+	-	-	-	+	-	-	-	+
	-	-	5		6	-	8		9	-	-	
	9	-	-		-	7	-		-	-	8	
	-	-	2		1	-	3		4	-	-	
+	-	-	-	+	-	-	-	+	-	-	-	+
	-	2	-		4	-	7		-	6	-	
	-	5	8		-	-	-		3	1	-	

という初期状態を持つsample-board-5という問題を

(examin sample-board-5)

と実行すると

初期盤面です。#0(root)

```
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----#-----#-----#-----#-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----#-----#-----#-----#-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . 2 3 #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----#-----#-----#-----#-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . 6 #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----#-----#-----#-----#-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | 4 . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----#-----#-----#-----#-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | 4 . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | 7 . 9 #
#=====#
```

*現在のノードは #0(root) です。

と、刈り込みを行った初期盤面が表示され、その直後にメニューが表示されます。初期設定ファイル NumberPlace-init.lisp に記載してあるメニューの初期設定値は「N)ormal」です。

メニューのプロンプトに対して「methods」あるいは「m」と入力しリターンキーをタイプする（以後は「リターンキーをタイプする」は省略）と数秒後に

```
=====
適用可能な手筋は次の通りです。
=====
```

```
#0 --> #1 (nil) 基本手筋 (finished)
#0 --> #2 (nil) 配置確定法 (finished)
#0 --> #3 (nil) ローカライゼーション (finished)
#0 --> #4 (nil) n国同盟 (finished)
#0 --> #5 (nil) nグリッド (finished)
#0 --> #6 (nil) Almost-Locked-Set (finished)
#0 --> #7 (nil) Grid-Based-Almost-Locked-Set (applied)
#0 --> #8 (nil) Advanced-Coloring (finished)
#0 --> #9 (nil) Nice-Loop (finished)
```

```
-----
*現在のノードは #0(root) です。
-----
```

と表示され、再びメニューが表示されます。上記の結果中の「#(シャープ)」記号の次の数字が「ノード番号」と呼ぶ特定の状態の盤面に付けられた番号です。初期状態のノード番号は「0（ゼロ）」に固定されています。ノード番号には覚えやすい「ラベル」を与えることもできます。#0ノードのラベルには「root」というラベルが与えられています。複数のノードに同名のラベルを与えることはできません。

```
#0 --> #1 (nil) 基本手筋 (finished)
```

というのはノード#0に対して基本手筋を適用した結果がノード#1であり、ノード#1の盤面は解答に至っている（finished）という意味です。ラベル名部分が「nil」となっているのは、ノード#1のラベルは存在していないことを表しています。ラベルを与えたり変更するコマンドは「Ch)ange」コマンドです。

```
#0 --> #7 (nil) Grid-Based-Almost-Locked-Set (applied)
```

は同様にノード#0の盤面にGrid-Based-Almost-Locked-Setを適用した結果の盤面がノード#7で、ラベルはなく、盤面はまだ最終的な解に至っていないことを表しています（applied）。

通常は表示されることはありませんが「finished」と「applied」の他に「inconsistent」と「unsolved」という状態も存在します。問題自体に矛盾が存在していることを検知した場合は「inconsistent」という状態マークを与えて、それ以上の手筋適用は行わないようにしています。「unsolved」はNumberPlace.lispが用意している試行錯誤法以外のすべての手筋で手を進められなかったということを示すマークです。この状態に至る解き筋（ルート）は「デッドルート」として「-->」という矢印部分を「==>」と置き換えて表示します。

途中でデッドルートが発生しても別のルートでは最終的な解に到達できる可能性があります。同じ手筋を使っても適用順序によって解ける場合と解けない場合が存在するという意味です。こういうときはデッドルートへの分岐点となったノードに「F)ind」コマンドで戻って、別のルートを探索します。

上記の例ではノード#7がappliedなので、まだ最終盤面に至っておらず手筋適用の余地があるということです。メニュー・プロンプトから

find 7

あるいは単に

7

と入力するとノード#7に移動し、現在のノードが#7となります。

```

Previous method : Grid-Based-Almost-Locked-Set
state is applied
Present board : #7(nil)
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . 2 3 #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# . . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . 6 #
# . 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#=====#=====#=====#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 7 . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | . 4 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | . . . #
# . . . | . . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | 7 . 9 #
#=====#=====#=====#=====#
*現在のノードは #7(nil) です。
-----

```

直前に適用された手筋が「Grid-Based-Almost-Locked-Set」であり状態は「applied」、現在のボードのノード番号が「#7」、ラベルはない (nil) ことを示しています。

この状態で再び

methods

と入力するとノード#7の盤面に対して適用可能な手筋の探索が行われて表示されます。

適用可能な手筋は次の通りです。

```
=====
#7 --> #10 (nil) 基本手筋 (finished)
#7 --> #11 (nil) 配置確定法 (finished)
#7 --> #12 (nil) ローカライゼーション (finished)
#7 --> #13 (nil) n国同盟 (finished)
#7 --> #14 (nil) nグリッド (finished)
#7 --> #15 (nil) Almost-Locked-Set (finished)
#7 --> #17 (nil) Advanced-Coloring (finished)
#7 --> #18 (nil) Nice-Loop (finished)
```

*現在のノードは #7(nil) です。

ここで「R)oute」 コマンドを入力すると現時点で判明しているノード#0からのすべてのルートを表示します。

```
*(SBCL/X86-64 21:28:37) Select menu : r
解法ルート[1] : #0(基本手筋)--> #1(finished)
解法ルート[2] : #0(配置確定法)--> #2(finished)
解法ルート[3] : #0(ローカライゼーション)--> #3(finished)
解法ルート[4] : #0(n国同盟)--> #4(finished)
解法ルート[5] : #0(nグリッド)--> #5(finished)
解法ルート[6] : #0(Almost-Locked-Set)--> #6(finished)
解法ルート[10] : #0(Grid-Based-Almost-Locked-Set)--> #7(基本手筋)--> #10(finished)
解法ルート[11] : #0(Grid-Based-Almost-Locked-Set)--> #7(配置確定法)--> #11(finished)
解法ルート[12] : #0(Grid-Based-Almost-Locked-Set)--> #7(ローカライゼーション)-->
#12(finished)
解法ルート[13] : #0(Grid-Based-Almost-Locked-Set)--> #7(n国同盟)--> #13(finished)
解法ルート[14] : #0(Grid-Based-Almost-Locked-Set)--> #7(nグリッド)--> #14(finished)
解法ルート[15] : #0(Grid-Based-Almost-Locked-Set)--> #7(Almost-Locked-Set)--> #15(finished)
解法ルート[17] : #0(Grid-Based-Almost-Locked-Set)--> #7(Advanced-Coloring)--> #17(finished)
解法ルート[18] : #0(Grid-Based-Almost-Locked-Set)--> #7(Nice-Loop)--> #18(finished)
解法ルート[8] : #0(Advanced-Coloring)--> #8(finished)
解法ルート[9] : #0(Nice-Loop)--> #9(finished)
```

*現在のノードは #7(nil) です。

解法ルート[10]から[18]はノード#0からノード#7にGrid-Based-Almost-Locked-Setで至った後、ノード#7に対して基本手筋を適用した結果がノード#10であり、ノード#10の盤面が最終的な解に至っていることを表しています (finished)。同様にノード#7に配置確定法を適用した結果がノード#11であり、やはり最終的な解に至っていることを表しています。以下、解法ルート[18]まで同様です。すべての

ルートでfinishedとなっているので、この問題はNumberPlace.lispが備えている試行錯誤法以外の9種類の手筋では全部で18種類の解き筋があったことになります。

この問題は非常にシンプルな例で、問題によっては数千通り以上の解き筋が存在する場合があります。

methodsコマンドの紹介はここまでにします。

任意の盤面で成立している手筋を使って手筋習得練習を行う

同じ問題を使って手筋習得練習を行う方法を紹介します。まずはノード#0に戻って初期盤面を練習材料にしましょう。

find o

または単に

0

と入力するとノード#0に戻ります。ここで「Gu)ess」 コマンドを入力します。

gu

または

guess

と入力します。

現在の盤面に対して適用可能な手筋の探索を行ってから刈り込み済みの対象盤面を表示します。今回は既にmethodsコマンドで適用できる手筋を探索済みだったので探索所要時間は0秒と表示されています。まだ適用できる手筋の探索が行われていなければCPUの速度にもよりますが1秒未満から数秒程度待つことがあります。

*(SBCL/X86-64 21:42:57) Select menu : [guess](#)

必要な情報を準備します...準備が整いました(0.0000 sec)。

```

=====
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . 2 . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . 6 #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . 2 . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . 2 3 #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . 6 #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | 1 2 3 #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . 6 #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | 4 . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | 7 . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . 2 . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | 4 . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | 7 . 9 #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
=====

```

0 *基本手筋 (0 1 2 3 4 5 6)

1 *ローカライゼーション (0 1)

2 *n国同盟 (0 1 2 3 4 5 6 7 8)

3 *nグリッド (0 1)

4 *Almost-Locked-Set (0 1 2 3)

5 *Grid-Based-Almost-Locked-Set (0 1)

6 *配置確定法 (0 1 2 3 4)

7 *Advanced-Coloring (0 1 2 3 4 5 6 7 8 9 10 11 12)

8 *Nice-Loop (0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16)

9 盤面についての情報を表示する。

10 セル式の条件にあうセルを彩色して表示する(盤面検討モード)。

[番号]+ "h)int" で該当手筋のヒント。

[番号]+ "e)xplanation" で該当手筋の解説。

[番号]+ "a)nsver" で該当手筋に対する解答入力。

Select number ("help"でコマンドのヘルプ、"quit"で中止)：

0番から8番まではNumberPlace.lispが実装している手筋名で、先頭に「*」が付いている手筋は実際に適用された手筋です。手筋名の後ろにカッコに囲まれて並んでいる数字は対象盤面に対して同一手筋が複数ヶ所で適用できた場合の成立回数です。8番のNice Loopは特に多く、0番から16番までの計17個のNice Loopが対象盤面で成立していることを示しています。

メニュー番号とコマンドの組み合わせで機能を実行

手筋名の前にある番号に4種類のコマンドを指定することで

- ・ 手筋習得練習 (手筋番号+a)nswer)
- ・ 番号に対応する手筋のヒント表示 (手筋番号+h)int)
- ・ 番号に対応する手筋の解説 (手筋番号+e)xplanation)
- ・ 番号に対応する手筋への解答 (手筋番号+a)nswer)

という4種類の機能が選べます。9番は現在の盤面に対する全体的な情報を表示します。

Select number ("help"でコマンドのヘルプ、"quit"で中止) : 9

 この盤面で成立している手筋は9種類。
 成立している個別の手筋数は61個あります。
 適用可能な手筋全体で候補が確定するセルは11箇所、
 候補数字を削除できるセルは38箇所あります。
 9+"show"で実際に使用されている手筋名の前に"*, 9+"hide"で表示なし。

「セル式」で条件に合うセルを彩色して表示する

メニューの10番は行・列・ブロック・ハウスを表わすrow, col, block, houseなどのキーワードと論理演算子のand, or, notを組み合わせで作成する「セル式」の条件を満たすセルを色付けして表示する機能を提供するメニューです。条件にあうセルに注目して考えたいときに使います。

Select number ("help"でコマンドのヘルプ、"quit"で中止) : 10

Enter Cell-Expression ("help"でセル式説明) : help

*盤面上のセル、行、列、ブロック、ハウスなどを論理式を使って指定し、彩色して表示します。

(row 2 4) と指定すると2行目と4行目全体を彩色して表示します。

(and (row 2 4) (col 3 5)) と指定すると2行目、4行目と3列目、5列目の交点のセルを彩色して表示します。

(or (row 2 4) (col 3 5)) と指定すると2行目、4行目と3列目、5列目全体を彩色して表示します。

(cell (2 3) (5 6) (8 9))と指定すると2行3列、5行6列、そして8行9列のセルを彩色して表示します。

「セル式」とはrow,col,cell,block,house,chain,candidate,determinedを構文要素としてand,or,notで構成される論理式のことです。条件に一致するセル・アドレスのリストを返します。

(or (row 3 5) (col 3 6)) と記述することで3行目と5行目、および3列目と6列目すべてのセルを表します。
(and (row 3 5) (col 3 6)) と記述すると3行目と5行目、および3列目と6列目の4つの交点のセルを表します。

(house (3 5)) は3行5列のセルを含むハウス全体のセルを表します。

この論理式は以下の文法に従って自由に記述できます。

```
[and]   ::= (and [exp]*) ;
[or]    ::= (or  [exp]*) ;
[not]   ::= (not [exp]*) = (not (or [exp-1] [exp-2]...[exp-n])) ;
```

```
[exp]   ::= ({and | or | not} [exp]*) | [cell] | [row] | [col] | [cell] | [block] | [chain] | [house] ;
```

```
[row]   ::= (row [i-exp]+) ;
[col]   ::= (col [i-exp]+) ;
[cell]  ::= (cell [exp]+) ;
[block] ::= (block [i-exp]+) ;
[chain] ::= (chain [exp]+) ;
[house] ::= (house [address]) ;
[address] ::= ([integer] [integer]) ;
[candidate] ::= ({cand|candidate}) | ({cand|candidate} {[i-exp]* | ([i-exp]+)})) ;
[determined] ::= ({det|determined}) | ({det|determined} {[i-exp]* | ([i-exp]+)})) ;
[i-exp] ::= 結果が整数となる式 ;
```

(not) は全てのセル・アドレスを返す。

(and) は空のセル・アドレスを表わす(一致するセル・アドレスなし)。

(or) は空のセル・アドレスを表わす(一致するセル・アドレスなし)。

* [candidate]式は指定された候補数字を含むセル・アドレスを返す。複数の候補数字が指定された場合は全ての候補数字を含むセル・アドレスを返す。

```
(candidate)                は確定値でない全てのセル・アドレスを返す。
(candidate (1 4))          は候補数字1と4の両方を含むセル・アドレスを返す。
(or (candidate (1)) (candidate (4)))  は候補数字1か4のいずれか、または両方を含むセル・アドレスを返す。
```

* [determined]式は指定されたセル・アドレスの内、確定値であるセル・アドレスを返す。

```
(determined)                は全ての確定値のセル・アドレスを返す。
(determined (3 7))          確定値が3か7である全てのセル・アドレスを返す。
(or (determined 3) (determined 7)) は上の式と同じ意味。
```

Notice. [s*]は直前の[s]のゼロ回以上の繰り返し、[s+]は直前の[s]の1回以上の繰り返し。「{A|B}」は「AかB」のどちらか一方。

プロンプトに対して例えば 3行目と6行目、そして3列目と7列目の和集合であるセルすべてに色を付けて（彩色して）表示したい場合は (or (row 3 6) (col 3 7)) と入力します。

```
Select number ("help"でコマンドのヘルプ、"quit"で中止) : 0 answer
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . B . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . B #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . B #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . B . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . B #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . B #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . B B #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . B #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . 2 3 | B B B #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . 2 . | . 2 3 | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . 6 | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . B #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#=====#=====#=====#=====#=====#=====#=====#=====#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | B . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#-----+-----+-----#-----+-----+-----#-----+-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . B . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | B . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | B . B #
#=====#=====#=====#=====#=====#=====#=====#=====#
列内で唯一の候補数字
Enter answer ("help","?","again" "quit") : 
```

Enter Cell-Expression ("help"でセル式説明) : (or (row 3 6) (col 3 7))

#	=====	#
#	1 2 . 1 . . B . . # 2 . # . B . . 2 . . 2 . #	
#	4 5 6 4 . 6 B . B # . 7 . . 3 . 4 . 6 # . B B 6 #	
#	 9 . . . # # 8 9 . . 9 #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	. 2 # 2 . . 2 . # 2 . #	
#	. 5 6 . 3 . . 7 . # . 8 . . 5 . . . 6 # . 1 . . 4 . . . 6 #	
#	 # # 9 #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	. B # B # . B B B #	
#	B B B . 8 . B . B # . 9 . B B . . 1 . # . B B . 7 . . . B #	
#	 # # #	
#	=====	#
#	1 . . 1 # 2 # 2 3 1 2 3 #	
#	4 . . 4 . . . 5 . # . 6 . 4 . . . 8 . # . 9 #	
#	7 . . 7 # # 7 . . #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	. . . 1 . . B . B # 2 . # . B . . 2 3 . . . #	
#	. 9 . 4 . 6 B . B # . 5 . . 7 . 4 . . # . . B 8 . #	
#	 # # #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	 # # #	
#	. . B . . B . 2 . # . 1 . . 9 . . 3 . # . 4 . . 5 . . . B #	
#	B B . B # # B . . #	
#	=====	#
#	 # # #	
#	. 3 . . 2 . . 9 . # . 4 . . 1 . . 7 . # . 8 . . 6 . . 5 . #	
#	 # # #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	 # # #	
#	4 . . . 5 . . 8 . # . 2 . . 6 . . 9 . # . 3 . . 1 . 4 . . #	
#	7 # # 7 . . #	
#	-----+-----+-----#-----+-----+-----#-----+-----+-----#	
#	1 . . 1 . . B . . # # . B . . 2 . . 2 . #	
#	4 . 6 4 . 6 B . B # . 3 . . 8 . . 5 . # 4 . . #	
#	7 . . 7 # # B 9 7 . 9 #	
#	=====	#

彩色されたセル内の候補数字が「B」となっているのはBlueで彩色されたという意味です。「カラーモード」の設定によって「B」ではなく本来の数字のまま表示することも可能です。ただし、その場合表示された盤面をコピー&ペーストするとテキスト・データだけがコピー&ペーストの対象となるので、ペースト後に色情報が失われます。

確定値はセル中央に表示しているので「B」とは表示しません。確定値の値が分からなくなるからです。確定値も数値に対応した位置に配置すれば「B」と置き換えることは可能ですが、セル中央に配置することでひと目で確定値と分かる分かりやすさを優先しています。

セル式は説明で表示されるBNF風の正規表現文法で規定される組み合わせの式を受け付けます。確定値だけを彩色して表示したり、特定の候補数字の組み合わせを持つセルだけを彩色して表示したりできます。盤面検討に役立ててください。彩色された色情報ごとコピーしたい場合はキャプチャ機能を使って下さい。

手筋を選択して問題に解答する

NumberPlace.lispが備える試行錯誤法以外の9つの手筋の一覧から解答したい手筋に対応した番号を選ぶことで手筋習得の練習を行えます。表示されている盤面に対して適用できる手筋名の前には「*」が付いています。「*」のない手筋番号を選んで何も起こらず、再び同じメニューが表示されます。

問題を選び、解答を始めるには「問題番号+a)answer」と入力します。例えば0番の基本手筋の最初の問題 (0) に解答するなら

0 answer

と入力します。「answer」は「a」のみでもOKです。したがって

0 a

でも「0 answer」と同じです。「a)answer」という場合は「answer」または「a」のいずれかで、大文字・小文字は無視するというルールはメニュー選択で説明したのと同じです。もちろん「e)xplanation」なども同じです。

「0 answer」と入力すると短いヒントと共に解答入力用のプロンプトが表れます。

盤面の直後に表示されているのがヒントです。注目すべきセルが彩色されて表示されています。この場合はブルーに彩色された列の中に列中で唯一の候補数字があると告げています。解答の入力方法はプロンプトに対して「help」と入力すると表示されます。

Enter answer ("help","?","again" "quit") : [help](#)

例えば、4行9列の内容(候補数字)が(1 2 3 7)のとき

セルの内容が「2, 3, 7のいずれでもない」が解答ならば(cannotbe (4 9) (2 3 7))と入力します。
セルの内容が「1」で確定ならば (mustbe (4 9) 1) と入力します。

本来、(cannotbe (4 9) (2 3 7))が正解の場合でも(mustbe (4 9) 1)も別解として許しています。

```
#=====#
# 1 2 . | 1 . . | 1 . . # . . . | . . . | . 2 . # . 2 . | . 2 . | . B . #
# 4 5 6 | 4 . 6 | 4 . 6 # . 7 . | . 3 . | 4 . 6 # . 5 6 | . . . | . . B #
# . . . | . . 9 | . . . # . . . | . . . | . . . # . . . | . 8 9 | . . B #
#-----+-----#-----+-----#-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . 2 . # . . . | . . . | . B . #
# . 5 6 | . 3 . | . 7 . # . 8 . | . 5 . | . . 6 # . 1 . | . 4 . | . . B #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . B #
#-----+-----#-----+-----#-----+-----#
# . 2 . | . . . | . . . # . . . | . 2 . | . . . # . 2 . | . . . | . B B #
# 4 5 6 | . 8 . | 4 . 6 # . 9 . | 4 5 . | . 1 . # . 5 6 | . 7 . | . . B #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----#-----+-----#-----+-----#
# 1 . . | 1 . . | . . . # . . . | . 2 . | . . . # . . . | . B B | B B B #
# 4 . . | 4 . . | . 5 . # . 6 . | 4 . . | . 8 . # . 9 . | . . . | . . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#-----+-----#-----+-----#-----+-----#
# . . . | 1 . . | 1 . 3 # . . . | . . . | . 2 . # . B . | . B B | . . . #
# . 9 . | 4 . 6 | 4 . 6 # . 5 . | . 7 . | 4 . . # . . B | . . . | . 8 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----#-----+-----#-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . . 6 | . . 6 | . 2 . # . 1 . | . 9 . | . 3 . # . 4 . | . 5 . | . . B #
# 7 8 . | 7 . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#-----+-----#-----+-----#-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# . 3 . | . 2 . | . 9 . # . 4 . | . 1 . | . 7 . # . 8 . | . 6 . | . 5 . #
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
#-----+-----#-----+-----#-----+-----#
# . . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | . . . #
# 4 . . | . 5 . | . 8 . # . 2 . | . 6 . | . 9 . # . 3 . | . 1 . | B . . #
# 7 . . | . . . | . . . # . . . | . . . | . . . # . . . | . . . | B . . #
#-----+-----#-----+-----#-----+-----#
# 1 . . | 1 . . | 1 . . # . . . | . . . | . . . # . 2 . | . 2 . | . B . #
# 4 . 6 | 4 . 6 | 4 . 6 # . 3 . | . 8 . | . 5 . # . . . | . . . | B . . #
# 7 . . | 7 . . | . . . # . . . | . . . | . . . # 7 . . | . . 9 | B . B #
#=====#
```

4行8列の内容が(2 3)のとき「2でない」は(cannotbe (4 8) (2))と書くのが正式ですが
(cannotbe (4 8) 2)と書くことも許しています。

同様に(mustbe (4 8) 3)を(mustbe (4 8) (3))と書くことも許容しています。

ただし(mustbe (9 9) (4 7 9))と書いた場合は2番目以降の候補数字(7 9)は切り捨てられます。

ナンバープレースの場合、セルに対して行う操作は「候補数字の削除」か「候補数字の確定」のみです。NumberPlace.lispでは

「**i**行**j**列の候補数字から**k l m**を削除する」を「**(cannotbe (i j) (k l m))**」と書きます。削除対象の候補数字がひとつだけの場合は非正規記法ですが「**(cannotbe (i j) k)**」と書くことも許しています。内部的には「**(cannotbe (i j) (k))**」と変換して扱っています¹。「**(cannotbe (i j) k l m)**」と書いてしまった場合は警告表示を行って、再入力促します。

「**i**行**j**列のセルを候補数字**k**で確定する」は「**(mustbe (i j) k)**」と書きます。確定値として指定できるのはひとつだけなので「**k**」はカッコで囲みません。非正規記法ですが誤ってカッコで囲んで「**(mustbe (i j) (k))**」と書いた場合も「**k**」がカッコで囲われていない場合と同じに扱っています。しかし「**(mustbe (i j) (k l m))**」は誤りです。この場合は「**l**」と「**m**」を切り捨てて「**(mustbe (i j) k)**」と同じに扱ったという警告表示を行います。

この問題の正解は「**(cannotbe (4 9) (2 3 7))**」です。プロンプトに解答を入力すると答え合わせが行われ合否の判定が行われます。

このケースではセル内のすべての候補数字は「**1 2 3 7**」の4つですから「**2 3 7でない**」というのは、結果だけ見ると「**1である**」と同じです。手筋によって得られる結果は論理的には「**(cannotbe (4 9) (2 3 7))**」ですが、「**(mustbe (4 9) 1)**」も正解として許しています。どちらの形式で解答を入力しても大丈夫です。

判定が正解か不正解かの2択でないのは複数の解が存在する場合があるからです。例えば3つの解が存在するケースで2つの解答を入力して2つとも正解だった場合、正答率は2/3です。この成績が合格か否かは合否ラインの設定によって決まります。合否ラインは初期設定ファイルNumberPlace-init.lispの中に書いてあります。

```
(cond
  ((equal (selected-user-level) 'novice-level)
    (grading-level 6/100)
  )
  ((equal (selected-user-level) 'middle-level)
    (grading-level 12/100)
  )
  ((equal (selected-user-level) 'senior-level)
    (grading-level 24/100)
  )
  ((equal (selected-user-level) 'advanced-level)
    (grading-level 48/100)
  )
  ((equal (selected-user-level) 'machine-level)
    (grading-level 96/100)
  )
)
```

¹ ユーザが使うセル・アドレスは1行1列から9行9列までのアドレスですが、内部的には0行0列から8行8列までのアドレスとして扱っています。これはCommon Lispの2次元配列が0行0列から始まるのに合わせているためです。ユーザからの入力と表示は自動的に変換しているので意識する必要はありません。

条件付き合否ラインはユーザ・レベルに応じて設定しています。Common Lispは分数をそのまま扱えるので合否ラインは $n/100$ という形式で設定してあります。この分数部分を望みの合否ラインに書き換えることで、合否ラインを変更できます。

最後に設定した合否ラインが有効なので、初期設定ファイルNumberPlace-init.lispの上記の条件付き設定の直後に

```
(grading-level 3/100)
```

と書き加えれば3/100以上の成績で合格となります。

入力プロンプトには「help」の他に「?」というヘルプ項目もあります。これは入力すべき解答数、設定されているユーザ・レベル、合格基準を表示する短いメッセージです。

```
Enter answer ("help","?","again" "quit") : ?
入力プロンプトの":"の数は入力すべき解答の数を表しています。
現在のレベル設定はmachine-levelです。
現在のレベル設定での合格基準は 3.0%以上です。
Enter answer ("help","?","again" "quit") :
```

「again」と入力するとヒント用の彩色された盤面を再度表示します。

「番号+h)int」で表示するのは「番号+a)nswer」で解答入力の前に表示される彩色された盤面付きのヒントと同じものです。実際に解答入力を求められる前に問題を検討したい場合に選びます。

「番号+e)xplanation」で表示される手筋の解説はNumberPlace.lisp内部に保存されている解説文章です。

処理系のプロンプトから「(help)」とだけ入力すると個別に使用できる関数の説明が表示されます。

「(help 'help)」と入力すると、更に別途文書化された項目一覧が表示されます。

```
> (help 'help)
(advanced-level als-show-all als-show-stat capital-address cell-expression color-mode
examin gb-als-show-all machine-level max-nice-length menu middle-level min-nice-length
n-grid-limit need-multiple-answer novice-level print-color-sample sel senior-level
set-parity set-unique-or-series tuples-limit fundamental localization n-tuples n-grid
almost-locked-set grid-based-almost-locked-set pattern-overlay-method advanced-coloring
nice-loop)
>
```

この一覧の中から手筋名に該当するシンボルを指定すると、その手筋についての解説文書が表示されます。例えば「(help 'fundamental)」と入力すると基本手筋に関する解説文書が表示されます。「番号+e)xplanation」で表示するのは「(help '手筋名)」と同じものです。

その他のメイン・メニューのメニュー項目は、コマンド名の説明を読み、試しに実行してみれば直ぐに意味が分かるものばかりです。実際に試してみてください。

解き筋の分岐を選択しながら検討する`m)ethods`コマンドと共によく使うのは`r)oute`コマンドです。判明している最新の解き筋の全ルートを表示します。

`lp)r`コマンドは現在の盤面をプリンタに印刷するコマンドです。外部コマンドの`lpr`コマンドを呼び出しています。一般的な`lpr`コマンドは`utf-8`文字コードに対応していないので「`paps`」というテキストをPostScriptファイルに変換するコマンドをフィルタとして使用しないと日本語部分の文字が化けます。`lp)r`コマンドは比較的使用頻度が高いと思われるので「`paps`」コマンドをインストールしておくことを強く推奨します。`paps`コマンドがなくても印刷自体は行えます。

その他、全体を通して共通しているのは各場所でのプロンプトには表示されていなくても「`quit`」コマンドが備わっていて`quit`すると直前のメニューに戻るということです。また「`quit`」の別名として「`q`」だけでも大丈夫なものも共通です。

2024-05-14 mail2daigo@gmail.com
Copyright 2024, GPL 3